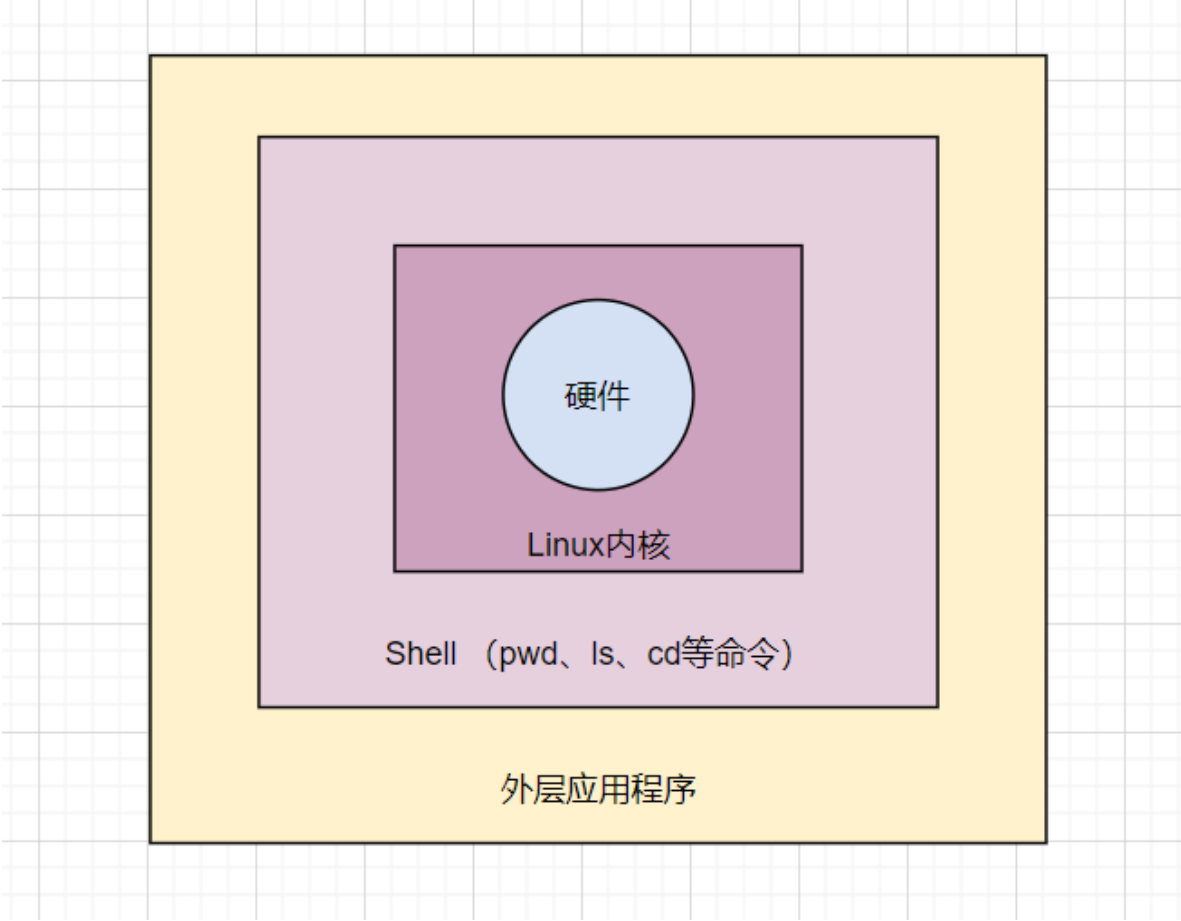
Linux Shell 教学文档

一、什么是 Shell

了解 Shell 的本质，首先要明确 Linux 系统的层级结构：**Linux 内核（Kernel）** 是系统的核心，负责管理硬件资源，但它不直接和用户交互；而**Shell** 是介于**用户**和**Linux 内核**之间的 "翻译官"，它接收用户输入的命令（比如 `ls`、`cd`），将其解析为内核能理解的指令，执行后再将结果返回给用户。



1.1、Shell 的分类

- 传统 Shell：`sh`（Bourne Shell），是最早的 Unix Shell
- 主流 Shell：`bash`（Bourne-Again Shell），是 `sh` 的增强版，兼容 `sh`，支持命令补全、历史记录、脚本编程等功能，**绝大多数 Linux 发行版默认使用 bash。**
- Dash，全称Debian Almquist Shell，是一种轻量级的Unix shell，1997年移植到Linux系统，并于2002年正式更名为DASH。**执行速度快、运行时依赖库少、体积小巧**
- 其他 Shell：`zsh`（功能丰富，适合终端美化）、`csh`（C 语言风格语法）等。

1.2、查看系统已安装的 Shell

```
cat /etc/shells
```

1.3、查看当前系统默认 Shell

- 查看系统脚本的Shell

```
ls -l /bin/sh
```

- 查看用户交互式的Shell

```
echo $SHELL
```

> 通过终端登录Ubuntu系统时，默认启动的交互式命令行解释器是 **Bash**
>
> 对于系统启动脚本和许多系统服务脚本,Ubuntu默认使用的Shell是 **Dash**,Ubuntu做出这种区分的原因在于Dash比Bash**更轻量、启动速度更快**，并且严格遵循POSIX标准，这有助于加速系统启动过程但Dash的功能相对Bash更精简，不支持Bash的一些扩展语法

总结：

Shell 既是**交互命令行工具**，也是**脚本编程语言**，我们常说的 "Shell 编程" 主要指使用 `bash` 编写脚本文件，批量执行一系列 Linux 命令。

二、Shell 脚本的运行

Shell 脚本是一个以 `.sh` 为后缀（后缀非强制，仅为便于识别）的文本文件，里面存放了一系列按顺序执行的 bash 命令和编程逻辑。

2.1. 脚本的基本格式

脚本文件**第一行必须指定解释器**，格式为：

```
#!/bin/bash
```

- `#!` 用于告诉系统该脚本使用哪个解释器（shell）来执行。
- 后续行是具体的命令或 Shell 编程逻辑，`#` 开头的行是注释（仅用于说明，不执行）。

2.2. 编写第一个 Shell 脚本

创建一个名为 `hello.sh` 的脚本文件，内容如下：

```
#!/bin/bash
# 第一个Shell脚本：输出欢迎信息
echo "Hello, Shell Programming!"
```

2.3. 脚本的 3 种运行方式

2.3.1、赋予执行权限，直接运行（推荐）

1. 赋予脚本执行权限

```
chmod +x hello.sh
```

2. 执行脚本（必须指定**相对路径** `./` 或**绝对路径**，不能直接写脚本名）

```
# 相对路径（当前目录下执行）
./hello.sh
# 绝对路径（示例，需替换为你的实际文件路径）
/home/user/hello.sh
```

2.3.2、使用 bash/sh 解释器直接运行（无需执行权限）

直接调用`bash`或`sh`解释器，将脚本文件作为参数传入，无需提前赋予执行权限：

```
# 使用bash解释器
bash hello.sh
# 使用sh解释器（bash兼容sh，适合跨环境）
sh hello.sh
```

2.3.3、使用`source`或`.`运行（在当前 Shell 环境执行）

前两种方式会创建一个**子 Shell 进程**来执行脚本，脚本中的变量、环境修改不会影响当前 Shell 环境；而`source`或`.`会在**当前 Shell 环境**中执行脚本，修改会直接生效。

```
# 两种写法等价
source hello.sh
. hello.sh
```

案例运行效果：

```
Hello, Shell Programming!
```

总结：

1. Shell 脚本第一行必须写`#!/bin/bash`指定 bash 解释器。
2. 推荐使用「赋予执行权限 + 直接运行」的方式，符合脚本的可执行属性。
3. `source`/`.`运行脚本会影响当前 Shell 环境，适合配置类脚本。

三、Shell 变量

> **Mobaxitem设置中文**：
>
> 1. 执行 su -
> 2. 执行 : echo "export LANG=en_US.UTF-8" >> ~/.bashrc
> 3. 执行 : echo "export LC_ALL=en_US.UTF-8" >> ~/.bashrc
> 4. 执行 : source ~/.bashrc
> 5. 断开连接重新连接Linux

Shell 变量是用于存储数据（字符串、数字等）的容器，无需提前声明类型，直接赋值即可使用，是 Shell 编程的基础。

3.1. 变量的分类

- **用户自定义变量**：用户自己创建、赋值、使用的变量，仅在当前 Shell 进程（或脚本）中有效（局部变量）。
- **环境变量**：系统预定义或用户通过`export`声明的变量，全局有效，可被子 Shell 继承（比如`PATH`、`HOME`、`USER`）。
- **系统预定义变量**：Shell 内置的变量，用于获取脚本运行的参数信息（比如`$0`、`$1`、`$#`、`$?`）。

3.2. 用户自定义变量的规则

定义变量: 变量名=变量值

1. 变量名由**字母、数字、下划线**组成，**不能以数字开头。**
2. 变量赋值时，`=`两边**不能有空格。**name=kate$grenn
3. 变量值包含空格、特殊字符时，需要用单引号`'`或双引号`"`包裹。
4. **单引号**：原样输出变量值，不解析变量、命令替换；**双引号**：解析变量、命令替换，保留空格。

3.3. 变量的操作

3.3.1、变量定义与使用

```
#!/bin/bash
# 1. 简单赋值
name="Zhang San"
age=25

# 两种使用方式
echo '姓名：$name，年龄：$age'
echo "姓名：${name}，年龄：${age}（推荐格式）"


# 3. 命令替换赋值（两种方式：`` 或 $()，推荐$()，可读性更强）
current_dir=$(pwd)
current_date=`date +%Y-%m-%d`
```

3.3.2、修改变量

直接重新赋值即可，无需额外命令。

```
#!/bin/bash
name="Zhang San"
echo "修改前：$name"
name="Li Si"
echo "修改后：$name"
```

3.3.4、删除变量

使用`unset`命令删除变量，删除后变量无法再使用。

```
#!/bin/bash
name="Zhang San"
echo "删除前：$name"
unset name
echo "删除后：$name"  # 输出为空
```

3.3.5、声明环境变量

环境变量：就是全局变量，在**所有shell进程**中都可以使用 。

局部变量： 用户自定义的变量，只能在当前sehll进程中使用 。

使用`export`命令将用户自定义变量转为环境变量，全局有效。

```
#!/bin/bash
export PATH=$PATH:/home/user/bin  # 向PATH添加自定义命令目录
export USER_NAME="Zhang San"      # 自定义环境变量
```

3.4. 系统预定义变量（常用）

变量	含义
`$0`	脚本的文件名（包含路径）
`$1~$9`	脚本运行时传入的第 1~9 个参数
`${10}`	脚本运行时传入的第 10 个及以上参数（必须用 {} 包裹）
`$#`	脚本传入的参数总个数
`$?`	上一条命令的执行返回状态（0 表示执行成功，非 0 表示执行失败）
`$*`	所有传入的参数（视为一个整体）
`$@`	所有传入的参数（视为独立个体，比`$*`更常用）

系统预定义变量案例：

创建`args_demo.sh`：

```
#!/bin/bash
echo "脚本文件名：$0"
echo "第1个参数：$1"
echo "第2个参数：$2"
echo "参数总个数：$#"
echo "所有参数（$*）：$*"
echo "所有参数（$@）：$@"
echo "上一条命令执行状态：$?"
```

运行脚本并传入参数：

```
chmod +x args_demo.sh
./args_demo.sh apple banana orange
```

运行效果：

```
脚本文件名：./args_demo.sh
第1个参数：apple
第2个参数：banana
参数总个数：3
所有参数（$*）：apple banana orange
所有参数（$@）：apple banana orange
上一条命令执行状态：0
```

总结:

1. 变量赋值 `=` 两边无空格，使用变量推荐 `${变量名}` 格式。
2. 单引号原样输出，双引号解析变量 / 命令，按需选择。
3. 系统预定义变量 `$1`（参数）、`$?`（执行状态）、`$#`（参数个数）是脚本开发高频使用的变量。

四、Shell 运算符

Shell 支持多种运算符，但**默认不支持直接的算术运算**，需要借助特定语法（`$(( ))`、`expr` 等）来实现，核心运算符分类如下：

4.1、算术运算符

用于**整数运算**（不支持小数），常用语法：`$(( 表达式 ))`（最简洁、推荐），`expr 表达式`（注意运算符两边有空格）。

运算符	含义	示例（$(()) 语法）
`+`	加法	`$(( 10 + 5 ))` → 15
`-`	减法	`$(( 10 - 5 ))` → 5
`*`	乘法	`$(( 10 * 5 ))` → 50
`/`	除法（整数除法，取商）	`$(( 10 / 3 ))` → 3
`%`	取余（模运算）	`$(( 10 % 3 ))` → 1
`=`	赋值	`a=$(( 10 + 5 ))` → a=15
`++`	自增	`$(( a++ ))`、`$(( ++a ))`
`--`	自减	`$(( a-- ))`、`$(( --a ))`

算术运算符案例:

```
#!/bin/bash
a=10
b=3

# 加法
echo "a + b = $(( a + b ))"
# 减法
echo "a - b = $(( a - b ))"
# 乘法
```

```
echo "a * b = $(( a * b ))"
# 除法
echo "a / b = $(( a / b ))"
# 取余
echo "a % b = $(( a % b ))"
# 复合运算
echo "a + b * 2 = $(( a + b * 2 ))"
echo "(a+b)*2 = $(( (a + b) * 2 ))"
# 自增
echo "a自增后（后置）: $((a++))"
echo "a自增后（前置）: $((++a))"
```

4.2、关系运算符

仅用于**整数比较**，通常和条件判断（`if`、`test`）配合使用，常用语法：在`[ ]`或`[[ ]]`中使用。

运算符	含义	示例（`[ $a -eq $b ]`）
`-eq`	等于（equal）	若 a=b，返回真
`-ne`	不等于（not equal）	若 a≠b，返回真
`-gt`	大于（greater than）	若 a>b，返回真
`-lt`	小于（less than）	若 a<b，返回真
`-ge`	大于等于（greater or equal）	若 a≥b，返回真
`-le`	小于等于（less or equal）	若 a≤b，返回真

关系运算符案例：

```
#!/bin/bash
a=10
b=3

if [ $a -eq $b ]
then
    echo "$a 等于 $b"
else
    echo "$a 不等于 $b"
fi

if [ $a -gt $b ]
then
    echo "$a 大于 $b"
else
    echo "$a 小于等于 $b"
fi
```

4.3、布尔运算符

用于逻辑判断（与、或、非），配合条件判断使用。

运算符	含义	适用场景	示例
`!`	非（取反）	`[ ]`、`[[ ]]`	`[ ! $a -eq $b ]`（a 不等于 b）
`-a`	与（同时成立）	`[ ]` 内部	`[ $a -gt 5 -a $a -lt 15 ]`
`-o`	或（任一成立）	`[ ]` 内部	`[ $a -lt 5 -o $a -gt 15 ]`
`&&`	与（同时成立）	`[[]]` 内部或命令之间	`[[ $a -gt 5 && $a -lt 15 ]]`
`||`	或（任一成立）	`[[]]` 内部或命令之间	`[[ $a -gt 5 || $a -lt 15 ]]`

布尔运算符案例：

```
#!/bin/bash
a=15

if [ ! $a -eq 15 ]
then
    echo "$a 不等于 15"
else
    echo "$a 等于 15"
fi

if [ $a -gt 5 -a $a -lt 20 ]
then
    echo "$a 在5--20之间"
else
    echo "$a 不在5--20之间"
fi

if [ $a -lt 5 -o $a -gt 20 ]
then
    echo "$a 不在5--20之间"
else
    echo "$a 在5--20之间"
fi

if [[ $a -gt 5 && $a -lt 20 ]]
then
    echo "$a 在5--20之间"
else
    echo "$a 不在5--20之间"
fi

if [[ $a -lt 5 || $a -gt 20 ]]
then
    echo "$a 不在5--20之间"
else
    echo "$a 在5--20之间"
fi
```

4.4、字符串运算符

用于字符串的比较、判断，常用语法：在`[ ]`或`[[ ]]`中使用。

运算符	含义	示例
`=`	字符串相等（`[ ]`中使用）	`[ "$str1" = "$str2" ]`
`==`	字符串相等（`[[ ]]`中推荐使用，支持通配符）	`[[ "$str1" == "$str2" ]]`
`=~`	正则匹配（`[[ ]]`中推荐使用）	`[[ $a =~ ^[0-9]+$ ]]`
`!=`	字符串不相等	`[ "$str1" != "$str2" ]`
`-z`	字符串长度为 0（空字符串）	`[ -z "$str" ]`
`-n`	字符串长度不为 0（非空字符串）	`[ -n "$str" ]`
`$`	字符串非空（直接使用变量，等价于`-n`）	`[ "$str" ]`

字符串运算符案例:

```
#!/bin/bash
str1="hello"
str2="world"
str3=""

if [ "$str1" = "$str2" ]
then
    echo "str1 等于 str2"
else
    echo "str1 不等于 str2"
fi

if [ -z "$str3" ]
then
    echo "str3 是空字符串"
fi

if [ -n "$str1" ]
then
    echo "str1 是非空字符串"
fi

str4="hello.pdf"
if [[ "$str4" == *.pdf ]]
then
   echo "str4是pdf文件"
else
   echo "str4不是pdf文件"
fi

x=10rrrr
if [[ "$x" =~ ^[0-9]+$ ]]
then
   echo "x是数字"
else
   echo "x不是数字"
fi

tel=136123456744448
```

```
if [[ "$tel" =~ ^1[3456789][0-9]{9}$ ]]
then
   echo "${tel}符合手机号规则"
else
   echo "${tel}不符合手机号规则"
fi
```

4.5、文件测试运算符

用于判断文件的类型、属性，是 Shell 脚本中操作文件的核心运算符，常用语法：在`[ ]`或`[[ ]]`中使用。

运算符	含义	示例
`-f`	判断是否为**普通文件**（非目录、非设备文件）	`[ -f "$file" ]`
`-d`	判断是否为**目录**	`[ -d "$dir" ]`
`-e`	判断文件 / 目录**是否存在**	`[ -e "$path" ]`
`-r`	判断文件**是否有读权限**	`[ -r "$file" ]`
`-w`	判断文件**是否有写权限**	`[ -w "$file" ]`
`-x`	判断文件**是否有执行权限**	`[ -x "$file" ]`
`-s`	判断文件**是否非空**（文件大小大于 0）	`[ -s "$file" ]`

文件测试运算符案例:

```
#!/bin/bash
file="./hello.sh"
dir="./test_dir"

if [ -e "$file" ]
then
    echo "$file 存在"
    if [ -f "$file" ]
    then
        echo "$file 是普通文件"
    fi
    if [ -x "$file" ]
    then
        echo "$file 有执行权限"
    fi
else
    echo "$file 不存在"
fi

if [ ! -d "$dir" ]
then
    echo "$dir 不存在，创建目录..."
    mkdir "$dir"
fi
```

总结:

1. 算术运算优先使用 `$(( 表达式 ))`，简洁高效。
2. 关系运算符仅支持整数，字符串比较用字符串运算符，文件判断用文件测试运算符。
3. `[ ]` 是标准语法，兼容性好；`[[ ]]` 是 bash 扩展语法，支持 `&&` / `||` 和通配符，推荐在 bash 脚本中使用。

五、Shell 的输入和输出

Shell 脚本的输入输出是与用户、系统交互的核心，主要包括**输出到终端**、**输入从终端读取**、**重定向（输出 / 输入到文件）**。

5.1、输出命令

5.1.1、`echo`（最常用）

用于向终端输出字符串、变量值，支持简单的转义字符。

- 基本语法：`echo [选项] [输出内容]`
- 常用选项：
- `-n`：输出后不换行（默认换行）。
- `-e`：支持转义字符（`\n` 换行、`\t` 制表符、`\b` 退格等）。
- 注意：转义字符需要放在双引号中，单引号会原样输出。

`echo` 案例：

```
#!/bin/bash
# 基本输出
echo "Hello, Shell!"
name="Zhang San"
echo "姓名：$name"

# -n 不换行输出
echo -n "中华人民"
echo "共和国万岁"

# -e 支持转义字符
echo -e "姓名：$name\n年龄：$age"
echo -e "这是我\t那是他"
echo -e "这是带退格的字符：abc\bdef"  # 输出abdef（\b删除c）
```

5.1.2、`printf`（格式化输出）

`printf` 是标准的格式化输出命令，比 `echo` 更规范、更灵活，支持格式化占位符，类似 C 语言的 `printf` 函数。

- 基本语法：`printf "格式化字符串" 变量1 变量2 ...`
- 常用格式化占位符：
- `%s`：字符串。
- `%d`：整数。
- `%f`：浮点数。
- `%n`：换行（`printf` 默认不换行，需手动添加）。
- `%10s`：占 10 个字符宽度，右对齐；`%-10s`：左对齐。

`printf` 案例：

```
#!/bin/bash
name="Zhang San"
age=25
score=98.5

# 基本格式化输出
printf "姓名：%s，年龄：%d，成绩：%f\n" "$name" "$age" "$score"

# 指定宽度，左对齐
printf "姓名：%-10s，年龄：%3d，成绩：%6.1f\n" "$name" "$age" "$score"

# 批量输出（遍历列表）
printf "水果：%s\n" "苹果" "香蕉" "橙子"
```

运行效果：

```
姓名：Zhang San，年龄：25，成绩：98.500000
姓名：Zhang San  ，年龄：  25，成绩：   98.5
水果：苹果
水果：香蕉
水果：橙子
```

5.2、输入命令：`read`

`read` 用于从终端读取用户输入，并将输入赋值给一个或多个变量，是脚本与用户交互的核心命令。

- 基本语法：`read [选项] [变量名1 变量名2 ...]`
- 常用选项：
- `-p`：输入前输出提示信息（最常用）。
- `-t`：设置输入超时时间（单位：秒），超时后脚本继续执行。
- `-n`：读取指定个数的字符，无需按回车结束。
- `-s`：隐藏输入内容（用于输入密码、敏感信息）。

read输入案例：

```
#!/bin/bash
# 1. 基本输入（无提示）
echo "请输入你的姓名："
read name

# 2. 带提示信息输入（-p）
read -p "请输入你的年龄：" age

# 3. 隐藏输入（-s，用于密码）
read -s -p "请输入你的密码：" password
echo -e "\n密码已输入（隐藏显示）"

# 4. 超时输入（-t，5秒超时）
if read -t 5 -p "请在5秒内输入你的城市：" city
then
    echo "你的城市：$city"
else
    echo -e "\n输入超时！"
```

```
fi

# 5. 读取指定字符数（-n，3个字符，无需回车）
read -n 3 -p "请输入3个字符：" char
echo -e "\n你输入的字符：$char"

# 输出所有输入内容
echo -e "\n汇总信息：姓名=$name，年龄=$age，城市=$city"
```

5.3、输入输出重定向

默认情况下，Shell 的输入来自终端（`stdin`，文件描述符 0），输出到终端（`stdout`，文件描述符 1），错误信息也输出到终端（`stderr`，文件描述符 2）。重定向可以将输入 / 输出转向**文件**，核心用于脚本的日志记录、结果保存。

> 每个运行中的进程（包括Shell命令和脚本）都会自动打开三个标准的数据流，它们由**文件描述符**（File Descriptor，简称FD）来标识
>
> - **0 - 标准输入 (stdin)**：默认从键盘获取输入。
> - **1 - 标准输出 (stdout)**：默认将输出显示在终端屏幕。
> - **2 - 标准错误 (stderr)**：默认将错误信息也显示在屏幕。

（1）输出重定向

符号	含义	示例
`>`	标准输出重定向（覆盖写），将命令输出写入文件，文件已存在则覆盖	`ls -l > file_list.txt`
`>>`	标准输出重定向（追加写），将命令输出写入文件，文件已存在则追加到末尾	`echo "新增内容" >> file_list.txt`
`2>`	标准错误重定向（覆盖写），将命令错误信息写入文件	`ls -l /nonexistent 2> error.log`
`2>>`	标准错误重定向（追加写）	`rm -rf /nonexistent 2>> error.log`
`>&` 或 `&>`	标准输出 + 标准错误重定向（同时写入同一文件）	`./script.sh &> all.log`（覆盖）、`./script.sh >> all.log 2>&1`（**追加，经典写法**）

（2）输入重定向

> 输入重定向: 系统从文件读内容，（read： 系统从终端读内容）

符号	含义	示例
`<`	标准输入重定向，将文件内容作为命令的输入	`cat < file.txt`（等价于 `cat file.txt`）
`<<`	此处文档（Here Document），将分界符之间的内容作为输入，常用 `EOF` 作为分界符	`cat << EOF > new_file.txt`

重定向案例：

```bash
#!/bin/bash
# 1. 标准输出覆盖写
echo "当前目录文件列表（覆盖写）" > file_list.txt
ls -l >> file_list.txt  # 追加写，避免覆盖上一行内容

# 2. 错误信息重定向
ls -l /nonexistent 2> error.log

# 3. 输出+错误同时重定向（追加到all.log）
./hello.sh >> all.log 2>&1

# 4. 此处文档（创建一个新文件，写入分界符EOF之间的内容）
cat << EOF > user_info.txt
姓名：Zhang San
年龄：25
城市：Beijing
EOF

echo "重定向操作完成，可查看 file_list.txt、error.log、user_info.txt"
```

总结：

1. 输出优先使用`echo`（简单）和`printf`（格式化），按需选择。
2. 与用户交互用`read`，`-p`（提示）、`-s`（隐藏密码）是高频选项。
3. 脚本日志记录优先使用`>>`（追加写）+ `2>&1`（输出 + 错误同时重定向），避免覆盖历史日志。

六、条件判断

Shell 的条件判断用于判断一个表达式是否成立，成立则执行后续逻辑，不成立则跳过或执行其他逻辑，核心配合`if`流程控制使用，有 3 种核心语法格式。

6.1、条件判断的 3 种语法格式

6.1.1、格式1:`test 表达式`

`test`命令用于判断表达式是否成立，成立则返回 0（真），不成立则返回非 0（假），语法简单但可读性差，较少使用。

```bash
#!/bin/bash
a=10
b=3

# 判断a是否大于b
if test $a -gt $b
then
    echo "$a 大于 $b"
fi

# 判断文件是否存在
if test -e "./hello.sh"
then
    echo "hello.sh 存在"
```

```
fi
```

6.1.2、格式 2：`[ 表达式 ]`

这是`test`命令的简化写法（`[`是`test`的别名），**最常用**，兼容性好，注意事项：

1. `[`和`]`两边**必须有空格**。
2. 表达式中的变量、字符串**推荐用双引号包裹**，避免空值、空格导致语法错误。
3. 逻辑与 / 或使用`-a`/`-o`，不支持`&&`/`||`。

格式 2 案例：

```
#!/bin/bash
file="./hello.sh"
str="hello"
a=15

# 1. 文件判断
if [ -f "$file" ] && [ -x "$file" ]  # 多个条件用&&连接（外部）
then
    echo "$file 是可执行普通文件"
fi

# 2. 字符串判断
if [ -n "$str" -a "$str" = "hello" ]  # 内部用-a（与）
then
    echo "str 是非空字符串，且值为hello"
fi

# 3. 数值判断
if [ $a -ge 10 -o $a -le 20 ]  # 内部用-o（或）
then
    echo "$a 在 10~20 之间"
fi
```

6.1.3、格式 3：`[[ 表达式 ]]`

bash 扩展语法，**推荐使用**，功能更强大，注意事项：

1. `[[`和`]]`两边**必须有空格**。
2. 支持`&&`/`||`逻辑与 / 或，可读性更强。
3. 支持字符串通配符（`*`、`?`）和正则表达式匹配（`=~`）。
4. 变量、字符串同样推荐用双引号包裹。

格式 3 案例：

```
#!/bin/bash
str="hello world"
file="./hello.sh"
a=18
tel=13912345678

# 1. 支持&&/||
if [[ $a -gt 10 && $a -lt 20 ]]
then
    echo "$a 在 10~20 之间"
fi
```

```
# 2. 支持通配符
if [[ "$str" == hello* ]]  # * 匹配任意字符
then
    echo "$str 以hello开头"
fi

# 3. 支持正则表达式（=~）
if [[ "$tel" =~ ^1[3-9][0-9]{9}$ ]]
then
    echo "$tel 符合手机号规则"
fi
```

6.2、条件判断的核心场景

条件判断的表达式本质就是第四部分的**关系运算符、字符串运算符、文件测试运算符**，核心场景总结：

1. **文件操作**：判断文件 / 目录是否存在、是否可执行、是否为普通文件（`-e` / `-f` / `-d` / `-x`）。
2. **用户输入验证**：判断字符串是否为空、是否符合指定格式（`-n` / `-z` / `==` / 正则）。
3. **数值比较**：判断数值大小、范围（`-eq` / `-gt` / `-ge` 等）。

总结：

1. 条件判断优先使用 `[[ 表达式 ]]`（功能强、可读性好），兼容场景用 `[ 表达式 ]`。
2. 牢记 `[ / [[`、`]] / ]` 两边有空格，变量 / 字符串用双引号包裹，避免语法错误。
3. 条件判断的核心是配合后续的 `if` 流程控制，实现分支逻辑。

七、流程控制

Shell 的流程控制用于实现脚本的分支、循环逻辑，核心包括**条件分支（if、case）和循环（for、while、until）**，注意：Shell 流程控制的关键字（`if`、`for`、`do` 等）后必须加 `;` 或换行，`done`、`fi`、`esac` 用于结束对应逻辑。

7.1、if 条件分支

根据条件判断结果执行不同的分支，分为**单分支、双分支、多分支。**

7.1.1、单分支（满足条件执行，不满足跳过）

```
语法：
if [[ 条件表达式 ]]
then
    满足条件时执行的命令
fi

# 简写（一行内完成，需要加;）
if [[ 条件表达式 ]]; then
    满足条件时执行的命令
fi
```

7.1.2、双分支（满足条件执行 A，不满足执行 B）

```
语法：
if [[ 条件表达式 ]]
then
    满足条件时执行的命令
else
    不满足条件时执行的命令
fi
```

7.1.3、多分支（多个条件判断，满足对应条件执行对应逻辑）

```
语法：
if [[ 条件表达式1 ]]
then
    满足条件1时执行的命令
elif [[ 条件表达式2 ]]
then
    满足条件2时执行的命令
elif [[ 条件表达式3 ]]
then
    满足条件3时执行的命令
else
    所有条件都不满足时执行的命令
fi
```

```
#!/bin/bash

read -p "请输入月份:" m

if [[ "$m" -eq 1 || "$m" -eq 2 || "$m" -eq 3  ]]
then
   echo "$m 属于第一季度"
elif [[ "$m" -eq 4 || "$m" -eq 5 || "$m" -eq 6 ]]
then
   echo "$m属于第二季度"
fi
```

if 案例（成绩等级判断）：

```
#!/bin/bash
read -p "请输入你的考试成绩（0-100）：" score

# 先判断是否为有效数字（避免非数字输入）
if ! [[ "$score" =~ ^[0-9]+$ ]] || [[ "$score" -lt 0 ]] || [[ "$score" -gt 100
]]
then
    echo "输入无效！请输入0-100之间的整数。"
    exit 1  # 退出脚本，返回非0状态码
fi

# 多分支判断成绩等级
if [[ "$score" -ge 90 ]]
then
    echo "成绩等级：优秀（A）"
elif [[ "$score" -ge 80 ]]
then
```

```
    echo "成绩等级：良好（B）"
elif [[ "$score" -ge 70 ]]
then
    echo "成绩等级：中等（C）"
elif [[ "$score" -ge 60 ]]
then
    echo "成绩等级：及格（D）"
else
    echo "成绩等级：不及格（E）"
fi
```

7.2、case 多分支匹配

当需要对一个变量进行多值匹配时，`case` 比 `if` 多分支更简洁、可读性更强，类似其他语言的 `switch` 语句。

```
语法：
case 变量名 in
    模式1)
        匹配模式1时执行的命令
        ;;  # 结束当前分支（类似break）
    模式2)
        匹配模式2时执行的命令
        ;;
    模式3|模式4)  # 多个模式匹配同一逻辑（用|分隔）
        匹配模式3或模式4时执行的命令
        ;;
    *)  # 匹配所有未匹配的模式（类似default）
        默认执行的命令
        ;;
esac  # 结束case语句（case倒写）
```

case 案例（用户指令处理）：

```
#!/bin/bash
read -p "请输入指令（start/stop/restart/status）：" cmd

case "$cmd" in
    start)
        echo "正在启动服务..."
        echo "服务启动成功！"
        ;;
    stop)
        echo "正在停止服务..."
        echo "服务停止成功！"
        ;;
    restart)
        echo "正在重启服务..."
        echo "服务重启成功！"
        ;;
    status)
        echo "当前服务状态：运行中"
        ;;
    *)
        echo "输入无效！支持的指令：start/stop/restart/status"
        ;;
```

```
esac
```

7.3、for 循环

`for` 循环是 Shell 中最常用的循环，用于遍历列表、数字范围，分为**列表遍历型**和**C 语言风格型**。

7.3.1、列表遍历型（推荐，可读性强）

```
语法：
for 变量名 in 列表1 列表2 列表3 ...  # 列表可以是直接值、变量、命令结果
do
    循环体命令（可使用$变量名获取当前遍历值）
done
```

列表遍历型案例：

```
#!/bin/bash
# 1. 遍历直接列表
echo "遍历水果列表："
for fruit in 苹果 香蕉 橙子 葡萄
do
    echo "当前水果：$fruit"
done

# 2. 遍历命令结果（用$()获取ls命令的输出）
echo -e "\n遍历当前目录下的.sh文件："
for file in $(ls *.sh)
do
    echo "当前脚本文件：$file"
done

# 3. 遍历变量列表
echo -e "\n遍历变量列表："
names="Zhang San Li Si Wang Wu"
for name in $names
do
    echo "当前姓名：$name"
done
```

7.3.2、C 语言风格型（适合数值循环，指定起始、结束、步长）

```
语法：
for (( 初始化表达式; 条件判断表达式; 迭代表达式 ))
do
    循环体命令
done
```

C 语言风格型案例：

```
#!/bin/bash
# 打印1--10数字
for (( i=1;i<=10;i++ ))
do
   echo "$i"
done
```

```
echo "==============="
# 打印1--10的奇数

for (( i=1;i<=10;i+=2 ))
do
   echo "$i"
done

echo "==============="

# 打印 1-10的偶数
for (( i=1;i<=10;i++ ))
do
   if [ "$(( $i%2 ))" == 0 ]
   then
      echo "$i"
   fi
done
```

```
#!/bin/bash
# 1. 计算1~100的求和
sum=0
for (( i=1; i<=100; i++ ))
do
    sum=$(( sum + i ))
done
echo "1~100的和：$sum"

# 2. 步长为2（遍历1~10的奇数）
echo -e "\n1~10的奇数："
for (( i=1; i<=10; i+=2 ))
do
    echo "$i"
done

# 3. 反向遍历（10~1）
echo -e "\n10~1反向遍历："
for (( i=10; i>=1; i-- ))
do
    echo "$i"
done

echo "计算5的阶乘:"
sum1=1
for (( i=5;i>=1;i-- ))
do
   sum1=$(( sum1*i ))
done

echo "5的阶乘是:${sum1}"
```

7.3.3、循环控制语句（break、continue）

- `break`：**跳出当前整个循环**，不再执行后续循环体，直接结束循环。

- `continue`：**跳过本次循环的剩余命令**，直接进入下一次循环（不结束整个循环）。

```
#!/bin/bash
# break 案例：遍历1~10，遇到5则跳出循环
echo "break 案例："
for (( i=1; i<=10; i++ ))
do
    if [[ "$i" -eq 5 ]]
    then
        break
    fi
    echo "当前数字：$i"
done

# continue 案例：遍历1~10，跳过5
echo -e "\ncontinue 案例："
for (( i=1; i<=10; i++ ))
do
    if [[ "$i" -eq 5 ]]
    then
        continue
    fi
    echo "当前数字：$i"
done
```

7.4、while 循环

`while` 循环先判断条件，**条件为真则执行循环体**，条件为假则退出循环，适合不确定循环次数的场景（比如用户输入验证、猜数字游戏）。

```
语法：
while [[ 条件表达式 ]]
do
    循环体命令
done
```

1. 打印1-10的数字
2. 计算1+2+3+...+100的和
3. 计算5 的阶乘

```
#!/bin/bash

echo "输出1-10的数字:"

num=1
while [[ num -le 10 ]]
do
    echo $num
    num=$(( num+1 ))
done

echo "==============="
x=1
```

```
while true
do
   echo $x
   x=$(( x+1 ))
   if [[ x -eq 11  ]]
   then
        break
   fi
done

echo "计算1+2+3+...+100的和"
count=1
sum=0
while [[ count -le 100 ]]
do
    sum=$(( $sum+$count ))
    count=$((count+1))
done

echo $sum

echo "计算5的阶乘:"
i=1
sum1=1
while [[ i -le 5 ]]
do
    sum1=$(( $sum1*$i ))
    i=$((i+1))
done

echo $sum1
```

while 案例（猜数字游戏）：

```
#!/bin/bash
# 生成1~100之间的随机数  RANDOM (0--32767)
target=$(( RANDOM  % 100 + 1 ))
# 统计猜数字的次数
count=0

echo "欢迎来到猜数字游戏！"
echo "游戏规则：猜1~100之间的整数，猜对为止。"

while true  # 无限循环（条件永远为真）
do
    read -p "请输入你的猜测：" guess
    count=$(( count + 1 ))

    # 验证输入是否为有效数字
    if ! [[ "$guess" =~ ^[0-9]+$ ]]
    then
        echo "输入无效！请输入整数。"
        continue  # 跳过本次循环，进入下一次
    fi
```

```
    # 判断猜测结果
    if [[ "$guess" -lt "$target" ]]
    then
        echo "猜小了！再试试更大的数。"
    elif [[ "$guess" -gt "$target" ]]
    then
        echo "猜大了！再试试更小的数。"
    else
        echo "恭喜你！猜对了！目标数字是$target"
        echo "你一共猜了$count次。"
        break  # 跳出循环，结束游戏
    fi
done
```

7.5、until 循环

`until` 循环与`while`循环相反，**先判断条件，条件为假则执行循环体，条件为真则退出循环**，使用场景较少，适合 "直到条件满足才停止" 的场景。

```
语法：
until [[ 条件表达式 ]]
do
    循环体命令
done
```

until 案例（计算 1~50 的求和，直到和大于 1000）：

```
#!/bin/bash
sum=0
i=1

until [[ "$sum" -gt 1000 ]]
do
    sum=$(( sum + i ))
    i=$(( i + 1 ))
done

echo "当和大于1000时，累加的最后一个数是：$(( i - 1 ))"
echo "最终的和是：$sum"
```

总结:

1. 分支逻辑：简单分支用`if`，多值匹配用`case`（更简洁）。
2. 循环逻辑：遍历列表用`for`列表型，数值循环用`for`C 语言型，不确定循环次数用`while`。
3. 循环控制：`break`跳出整个循环，`continue`跳过本次循环，按需使用。

八、Shell 函数

Shell 函数用于将一段**重复使用**的**代码（功能）**封装成一个**命名**模块，便于调用、维护，减少代码冗余，类似其他语言的函数 / 方法。

8.1. 函数的定义

Shell 函数有两种定义格式，功能等价，推荐格式 2（可读性更强）。

```
# 格式1：简化格式
函数名() {
    函数体命令（一系列Shell命令、逻辑）
    # 返回0 代表执行正确  非0 是执行错误  不返回任何数字  代表代码执行结束
    # 如果向返回其他的数据，  使用 echo 进行返回
    [return 返回值]  # 可选，返回0~255之间的整数，默认返回最后一条命令的执行状态
}

# 格式2：完整格式（用function关键字声明，推荐）
function 函数名() {
    函数体命令
    [return 返回值]
}
```

```
#1.定义一个函数
function sayHello(){

   echo "hello world!"
}
#.调用函数
sayHello
```

8.2、函数的调用

直接在脚本中写**函数名**即可调用，若函数需要参数，调用时直接在函数名后跟上参数（用空格分隔），无需在函数定义时声明参数。

```
# 无参函数调用
函数名

# 有参函数调用
函数名 参数1 参数2 参数3 ...
```

8.3、函数的参数

函数内部获取参数的方式与脚本获取命令行参数一致，使用`$1`、`$2`、`$#`、`$@`等系统预定义变量：

- `$1`：函数的第 1 个参数，`$2`：函数的第 2 个参数，以此类推。
- `$#`：函数的参数总个数。
- `$@`：函数的所有参数（独立个体）。

```
#1.定义一个函数
function sayHello(){

   echo "接收到的参数个数是:$#"
   echo "接收到的参数是:$@"
   echo "hello $1,$2"
}
#.调用函数
sayHello lucy  lily
```

8.4、函数的返回值

Shell 函数有两种返回值方式，按需选择：

1. `return` **返回值**：仅支持返回**0~255**之间的整数，0 表示成功，非 0 表示失败，适合表示函数执行状态，通过`$?`获取返回值。

   ```
   #1.定义一个函数

   function test(){
      ls -l ./dddd  # 当前执行失败 返回非0 验证执行成功 把 ./dddd  去掉
   }
   # 调用函数
   test
   echo "test函数的返回值是:$?"
   ```

   ```
   #1.定义一个函数

   function test(){
      ls -l ./dddd  # 当前执行失败 返回非0 验证执行成功 把 ./dddd  去掉

      if [[ "$?" == 0 ]]
      then
          return 0
      else
          return 1
      fi
   }
   # 调用函数
   test
   echo "test函数的返回值是:$?"
   ```

2. `echo` **输出返回值**：支持返回任意类型数据（字符串、数字、复杂结果），通过`$(函数名)`捕获返回值，是获取函数执行结果的常用方式。

   ```
   #1.定义一个函数
   function add(){

     sum=$(( $1+$2 ))
     echo "$sum"
   }

   # 调用函数
   ```

```
add 10 20

# 接收add函数的返回值
result=$(add 100 200)
echo "接收到add函数的返回值是:$result"
```

8.5、函数的变量作用域

- **全局变量**：默认情况下，函数内外定义的变量都是全局变量，函数内部可以修改全局变量，函数外部也可以访问函数内部定义的变量。
- **局部变量**：在函数内部用`local`关键字定义的变量，仅在函数内部有效，外部无法访问，避免与全局变量冲突，**推荐在函数内部使用局部变量。**

```
#1.定义一个函数
global_count=100 # 全局变量 在函数内外都可以使用

function test(){
   fun_count=200  # 全局变量
   local test_count=300  # 局部变量 只能在当前功函数内部使用

   echo "test函数内打印:$global_count,$fun_count,$test_count"
}

#函数调用
test

echo "test函数外打印:$global_count,$fun_count,$test_count"
```

案例：

8.5.1、案例 1：无参函数（计算 1~100 的和）

```
#!/bin/bash
# 定义函数：计算1~100的和
function sum_1_to_100() {
    local sum=0  # 局部变量，仅函数内部有效
    for (( i=1; i<=100; i++ ))
    do
        sum=$(( sum + i ))
    done
    echo "$sum"  # 用echo返回结果
}

# 调用函数，捕获返回值
result=$(sum_1_to_100)
echo "1~100的和：$result"
```

8.5.2、案例 2：有参函数（计算两个数的和）

```
#!/bin/bash
# 定义函数：计算两个数的和与积
function calc() {
    # 验证参数个数是否为2
    if [[ $# -ne 2 ]]
```

```
    then
        echo "错误：请传入2个整数参数！"
        return 1  # 返回非0状态码，表示执行失败
    fi

    # 验证参数是否为整数
    local a=$1
    local b=$2
    if ! [[ "$a" =~ ^[0-9]+$ && "$b" =~ ^[0-9]+$ ]]
    then
        echo "错误：请传入整数参数！"
        return 1
    fi

    # 计算并返回结果（用空格分隔和与积）
    local add=$(( a + b ))
    echo "$add"
}

# 调用函数
read -p "请输入两个整数（用空格分隔）：" num1 num2
add=$(calc $num1 $num2)

# 判断函数执行是否成功（$?为0表示成功）
if [[ $? -eq 0 ]]
then
    echo "$num1 + $num2 = $add"
fi
```

8.5.3、案例 3：递归函数（计算 n 的阶乘）

```
#!/bin/bash
# 定义递归函数：计算n的阶乘（n! = n * (n-1)!）
function factorial() {
    local n=$1

    # 基线条件：n=0或n=1时，阶乘为1
    if [[ "$n" -eq 0 ]] || [[ "$n" -eq 1 ]]
    then
        echo 1
        return
    fi

    # 递归调用：n! = n * (n-1)!
    local prev=$(factorial $(( n - 1 )))
    echo $(( n * prev ))
}

# 调用函数
read -p "请输入一个非负整数：" num
if ! [[ "$num" =~ ^[0-9]+$ ]]
then
    echo "输入无效！请输入非负整数。"
    exit 1
fi

result=$(factorial $num)
```

```
echo "$num 的阶乘：$result"
```

总结：

1. 函数定义推荐使用 `function 函数名()` 格式，代码可读性更强。
2. 函数参数通过 `$1`、`$2` 获取，返回复杂结果用 `echo`，获取执行状态用 `return`。
3. 函数内部优先使用 `local` 定义局部变量，避免污染全局变量。
4. 递归函数需设置明确的基线条件，避免无限递归。

> 补充： 两个内置函数
>
> 1. dirname :
>
> 用于从完整路径中**提取目录部分**（即去掉最后的文件名）
>
> 2. basename:
>
> 从给定的文件路径字符串中剥离目录部分，仅返回末尾的文件名或目录名

```
full_path="/home/scripts/fun.sh"

dir_path=$(dirname "$full_path")
file_path=$(basename "$full_path")
file_name=$(basename "$full_path" .sh)
echo "目录:$dir_path"
echo "文件:$file_path"
echo "文件名:$file_name"
```

九、综合案例：文件备份脚本

9.1、案例需求

编写一个实用的文件 / 目录备份脚本，满足以下需求：

1. 支持指定要备份的源文件 / 目录和备份存储目录。
2. 备份文件按「源名称*年月日*时分秒.tar.gz」命名，避免覆盖。
3. 自动判断源文件 / 目录是否存在，不存在则退出并提示。
4. 自动判断备份目录是否存在，不存在则创建。
5. 支持压缩备份（使用 `tar` 命令），并输出备份进度信息。
6. 支持删除备份目录下 7 天前的旧备份文件，释放磁盘空间。
7. 备份完成后，输出备份结果（成功 / 失败）和备份文件路径。

> tar ： 打包压缩 解包 解压缩

```
tar -zcvf 压缩文件名 被压缩文件
tar -zcvf mydata.tar.gz /home/user/data > /dev/null 2>&1
#  > /dev/null 2>&1 把命令执行的输出（正确还是错误 重定向到 /dev/null上，实际上就是丢弃）
```

9.2、案例脚本（backup.sh）

```
#!/bin/bash
# 文件名：backup.sh
# 功能：文件/目录备份脚本，支持压缩、自动创建备份目录、删除7天前旧备份

# 定义全局变量（配置区）
BACKUP_SUFFIX=$(date +%Y%m%d_%H%M%S)  # 备份文件后缀（年月日_时分秒）
OLD_BACKUP_DAYS=7  # 删除7天前的旧备份

# 定义函数：输出脚本使用帮助
function show_help() {
    echo "使用方法：$0 [源文件/目录] [备份存储目录]"
    echo "示例：$0 /home/user/data /home/user/backup"
    echo "功能：将源文件/目录压缩备份到指定存储目录，自动删除7天前的旧备份。"
    exit 1
}

# 定义函数：执行备份操作
function do_backup() {
    local src_path=$1
    local backup_dir=$2

    # 1. 获取源文件/目录的基本名称（用于备份文件名）
    local src_name=$(basename "$src_path")
    local backup_file="${backup_dir}/${src_name}_${BACKUP_SUFFIX}.tar.gz"

    # 2. 执行压缩备份（-z：gzip压缩，-c：创建归档，-f：指定归档文件，-v：详细输出）
    echo "开始备份：$src_path → $backup_file"
    if tar -zcvf "$backup_file" "$src_path" > /dev/null 2>&1
    then
        echo "备份成功！备份文件路径：$backup_file"
        return 0
    else
        echo "备份失败！请检查源文件/目录权限和备份目录空间。"
        rm -f "$backup_file"  # 删除失败的备份文件
        return 1
    fi
}

# 定义函数：删除旧备份
function delete_old_backup() {
    local backup_dir=$1
    local src_name=$(basename "$2")

    echo "开始删除${OLD_BACKUP_DAYS}天前的旧备份文件..."
    # 查找备份目录下的对应源名称备份文件，删除7天前的文件
    find "$backup_dir" -name "${src_name}_*.tar.gz" -type f -mtime
+${OLD_BACKUP_DAYS} -delete

    if [[ $? -eq 0 ]]
    then
        echo "旧备份文件删除完成！"
    else
        echo "旧备份文件删除失败！"
        return 1
    fi
```

```
}

# 主逻辑开始
# 1. 验证参数个数（必须传入2个参数）
if [[ $# -ne 2 ]]
then
    echo "错误：参数个数不正确！"
    show_help
fi

# 2. 赋值参数变量
SRC_PATH=$1
BACKUP_DIR=$2

# 3. 验证源文件/目录是否存在
if ! [[ -e "$SRC_PATH" ]]
then
    echo "错误：源文件/目录 $SRC_PATH 不存在！"
    exit 1
fi

# 4. 验证并创建备份目录
if ! [[ -d "$BACKUP_DIR" ]]
then
    echo "备份目录 $BACKUP_DIR 不存在，正在创建..."
    if mkdir -p "$BACKUP_DIR"
    then
        echo "备份目录创建成功！"
    else
        echo "错误：备份目录 $BACKUP_DIR 创建失败！"
        exit 1
    fi
fi

# 5. 执行备份操作
if ! do_backup "$SRC_PATH" "$BACKUP_DIR"
then
    exit 1
fi

# 6. 删除旧备份
delete_old_backup "$BACKUP_DIR" "$SRC_PATH"

# 7. 脚本执行完成
echo "==================== 备份任务全部完成 ===================="
exit 0
```

9.3、案例使用说明

1. 脚本权限配置：

```
chmod +x backup.sh
```

2. 脚本运行示例：

```
./backup.sh /home/user/data /home/user/backup
```

3. 脚本运行效果：
   - 若备份目录不存在，自动创建。
   - 生成备份文件：`data_20260204_103000.tar.gz`。
   - 自动删除 `/home/user/backup` 目录下 7 天前的 `data_*.tar.gz` 文件。
   - 输出清晰的进度信息和最终结果。

9.4、案例知识点整合

该脚本整合了 Shell 编程的核心知识点，包括：

1. 变量定义与使用（全局变量、局部变量）。
2. 函数封装与调用（`show_help`、`do_backup`、`delete_old_backup`）。
3. 条件判断（`-e`、`-d` 判断文件 / 目录是否存在）。
4. 流程控制（`if` 分支、参数验证）。
5. 输入输出（`echo` 提示信息、重定向 `>/dev/null 2>&1` 隐藏冗余输出）。
6. Linux 命令调用（`tar` 压缩、`mkdir` 创建目录、`find` 删除旧文件）。
7. 脚本参数处理（`$1`、`$2`、`$#`）。

总结：

1. 该综合案例是实用的生产环境脚本，可直接修改配置（`OLD_BACKUP_DAYS`）适配实际需求。
2. 脚本的核心思想是「封装复用、容错处理」，通过函数减少冗余代码，通过多轮验证避免执行失败。
3. 掌握该案例后，可扩展更多功能（如邮件通知、备份日志记录、增量备份等）。

十、整体总结

本课程覆盖了 Shell 编程的核心知识点，从基础概念到综合实战，新手可按照章节顺序逐步学习，重点掌握：

1. Shell 脚本的基本格式和运行方式，牢记 `#!/bin/bash` 和 `chmod +x`。
2. 变量、运算符、输入输出是 Shell 编程的基础，需熟练掌握常用语法和避坑点。
3. 条件判断（`[[ ]]`）和流程控制（`if`、`for`、`while`）是实现脚本逻辑的核心。
4. 函数封装可提高代码的复用性和可维护性，是复杂脚本开发的必备技能。
5. 综合案例是知识点的整合应用，多动手修改、扩展脚本，能快速提升 Shell 编程能力。

Shell 编程的核心是「解决 Linux 系统的自动化问题」，多实践、多总结，才能熟练掌握各类场景的脚本开发技巧。